UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Análisis de Sistemas II — Código 037



PROYECTO FINAL

Control de inventario y préstamos de laboratorio

Automatización con n8n, PostgreSQL y Docker en un entorno local

Documento técnico

Autora: Irma Yaritza Katiuxca Jagan Xol

Carné: 4090-23-9748

Docente: Ing. Edgar Raúl Molina Rey

Sección: Unica

Modalidad: Individual

Cobán, Alta Verapaz — 26 de Octubre de 2026

Índice

1. Introducción 3
2. Planteamiento del problema 3
2.1 Situación actual 3
2.2 Problema identificado 4
2.3 Objetivo general 4
2.4 Objetivos específicos 4
2.5 Alcance y limitaciones 5
3. Arquitectura local 5
3.1 Componentes 5
3.2 Decisiones de diseño 6
4. Modelo de datos 6
4.1 Reglas de integridad 7
5. Descripción de los workflows 7
5.1 WF1 – Ingesta de préstamos 9
5.2 WF2 – Procesamiento de préstamos 10
5.3 WF3 – Salida de reportes 12
5.4 WF4 – Manejo de errores 13
5.5 Resumen de nodos utilizados 14
6. Manejo de errores y de los datos 15
6.1 Errores esperados 15
6.2 Errores inesperados 15
6.3 Consistencia de la información 15
7. Consideraciones de seguridad 16
8. Pruebas realizadas 17
9. Instalación y reproducibilidad 19
12. Conclusiones 25
12.1 Trabajo futuro 26
13. Referencias y recursos de apoyo 26

1. Introducción

Este documento describe el análisis, el diseño y la implementación de un sistema de control de inventario y préstamos para un laboratorio, desarrollado como Proyecto Final del curso Análisis de Sistemas II. El sistema automatiza el proceso completo de un préstamo de equipo: la recepción de la solicitud, la validación de los datos, la verificación del stock disponible, el registro del préstamo, la actualización del inventario y la detección diaria de préstamos vencidos.

La solución se construyó con n8n como motor de automatización, PostgreSQL como base de datos y Docker como plataforma de despliegue, todo en un entorno 100 % local. Como interfaz para los usuarios se desarrolló además una aplicación web con Spring Boot y Thymeleaf, respaldada por 82 pruebas unitarias automatizadas con JUnit, Mockito y JaCoCo. La ciberseguridad se trató como un eje transversal del diseño: cada componente incorpora al menos una medida de protección sobre las credenciales o sobre los datos que maneja.

El código fuente, los workflows exportados, los casos de prueba y los diagramas están disponibles en el repositorio público:

https://github.com/kathyjaganx/proyecto-final-as2-inventario

2. Planteamiento del problema

2.1 Situación actual

En muchos laboratorios universitarios los préstamos de equipo (laptops, proyectores, multímetros, kits de electrónica, cables) se controlan en cuadernos o en hojas de cálculo que se llenan a mano. Este método depende por completo de la persona encargada: no valida que los datos estén completos, no impide prestar más unidades de las que existen y no avisa cuando un equipo no se devuelve a tiempo.

2.2 Problema identificado

El control manual produce cuatro problemas principales:

- **Inconsistencia del inventario:** el stock registrado no coincide con el real porque los préstamos y las devoluciones no siempre se anotan.
- **Datos incompletos o erróneos:** carnés mal escritos, nombres ilegibles o fechas de devolución imposibles.
- **Equipo no devuelto:** no hay un mecanismo que detecte a tiempo los préstamos vencidos.
- **Falta de trazabilidad:** no queda registro de quién solicitó qué, cuándo, ni por qué se rechazó una solicitud.

2.3 Objetivo general

Automatizar el proceso de préstamos de equipo de laboratorio mediante workflows de n8n en un entorno local con Docker y PostgreSQL, garantizando la validación de los datos, la consistencia del inventario, la trazabilidad de todas las operaciones y la protección de las credenciales y los datos personales de los estudiantes.

2.4 Objetivos específicos

1. Recibir las solicitudes de préstamo mediante un webhook protegido con una clave de acceso y validar sus datos antes de procesarlas.
2. Registrar el préstamo y descontar el stock en una sola operación atómica, de modo que el inventario nunca quede inconsistente.
3. Detectar diariamente los préstamos vencidos y generar un reporte verificable en la base de datos y en un archivo CSV.
4. Registrar toda operación en una bitácora de auditoría y todo fallo en una tabla de errores.
5. Ofrecer una aplicación web que envíe las solicitudes al sistema y verificar su funcionamiento con pruebas unitarias automatizadas que cubran al menos el 80 % del código.

2.5 Alcance y limitaciones

El sistema cubre la solicitud y el registro de préstamos, el control del stock, la detección de vencimientos, la auditoría y el manejo de errores. Quedan fuera del alcance de esta versión el registro de devoluciones (la base de datos ya incluye el campo fecha_devolucion_real y el estado DEVUELTO para incorporarlo después) y el envío de notificaciones por correo.

3. Arquitectura local

3.1 Componentes

La solución se despliega con Docker Compose en una sola computadora. El archivo docker-compose.yml define dos contenedores y dos volúmenes persistentes:

Componente	Tecnología	Función
app-prestamos	Spring Boot 4.1 + Thymeleaf (puerto 8080)	Aplicación web donde se registran los préstamos. Envía cada solicitud al webhook del WF1 con la clave X-API-Key y muestra el resultado.
inventario_n8n	n8n (puerto 5678)	Ejecuta los cuatro workflows. Guarda su configuración y sus credenciales cifradas en el volumen n8n_data.
inventario_db	PostgreSQL 16 (puerto 5432)	Almacena el inventario, los préstamos, la auditoría y los errores en el volumen postgres_data.
Carpeta reportes/	Volumen compartido	Recibe los archivos CSV que genera el WF3. Es la única carpeta a la que n8n tiene acceso.
Archivo .env	Variables de entorno	Contiene las credenciales. Docker Compose las inyecta en los contenedores y Git lo excluye del repositorio.

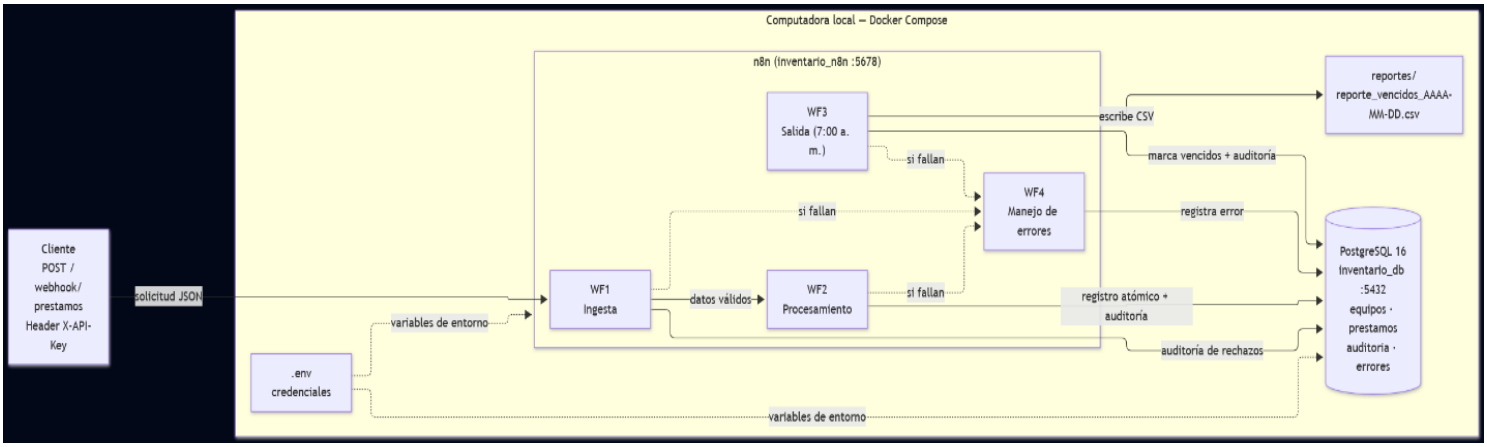


Figura 1. Diagrama de arquitectura del sistema.

3.2 Decisiones de diseño

- **Docker Compose:** permite levantar todo el entorno con un solo comando y que cualquier persona lo reproduzca en otra computadora.
- **PostgreSQL en lugar de archivos:** ofrece transacciones y restricciones de integridad, indispensables para que el stock sea siempre correcto.
- **Separación en cuatro workflows:** cada workflow tiene una sola responsabilidad (ingesta, procesamiento, salida y errores), lo que facilita probarlos y mantenerlos por separado.
- **Comunicación interna por nombre de servicio:** n8n se conecta a la base de datos con el host postgres, dentro de la red privada de Docker.
- **Zona horaria America/Guatemala:** configurada en ambos contenedores para que las fechas de auditoría y los cálculos de vencimiento correspondan a la hora local.

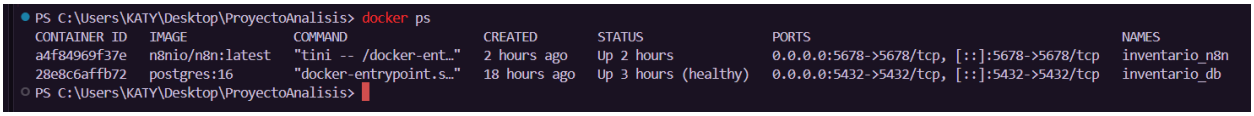


Figura 2. Contenedores del proyecto en ejecución.

4. Modelo de datos

El script db/init.sql crea las cuatro tablas al iniciar PostgreSQL por primera vez y carga cinco equipos de prueba.

Tabla	Campos principales	Propósito
equipos	id_equipo, codigo (único), nombre, categoria, stock_total, stock_disponible, activo	Inventario del laboratorio.
prestamos	id_prestamo, carne, nombre_estudiante, id_equipo (FK), cantidad, fecha_prestamo, fecha_devolucion_esperada, fecha_devolucion_real, estado	Registro de cada préstamo y su estado: ACTIVO, DEVUELTO o VENCIDO.
auditoria	id_auditoria, fecha, workflow, accion, resultado, detalle (JSONB)	Bitácora de ejecución de todas las operaciones, con resultado EXITO, RECHAZADO o ERROR.
errores	id_error, fecha, workflow, nodo, mensaje, id_ejecucion, datos (JSONB)	Fallos capturados por el WF4.

4.1 Reglas de integridad

La base de datos hace cumplir por sí misma las reglas del negocio, aunque un workflow tuviera un error:

- chk_stock_coherente: el stock disponible nunca puede ser mayor que el stock total ni menor que cero.
- chk_fechas: la fecha de devolución esperada no puede ser anterior a la fecha del préstamo.
- La cantidad prestada debe ser mayor que cero y el estado solo admite los valores ACTIVO, DEVUELTO o VENCIDO.
- La llave foránea id_equipo impide registrar préstamos de equipos inexistentes.
- Los índices sobre estado y fecha_devolucion_esperada aceleran la búsqueda diaria de vencidos.

Los campos detalle y datos usan el tipo JSONB, que permite guardar información estructurada distinta en cada evento (los datos de la solicitud, la lista de errores o el reporte completo) sin cambiar el esquema.

5. Descripción de los workflows

El sistema está formado por cuatro workflows que cumplen los tres tipos exigidos (ingesta, procesamiento y salida) más uno dedicado al manejo de errores.

Workflow	Tipo	Qué hace
WF1 – Ingesta de préstamos	Ingesta	Recibe la solicitud por un webhook protegido, valida los datos y decide si la rechaza o la envía a procesamiento.
WF2 – Procesamiento de préstamos	Procesamiento	Verifica el stock, registra el préstamo y descuenta el inventario en una sola operación.
WF3 – Salida de reportes	Salida	Cada día a las 7:00 a. m. marca los préstamos vencidos y genera un reporte en la base de datos y en un archivo CSV.
WF4 – Manejo de errores	Errores	Se activa cuando falla cualquier otro workflow y registra el error.

La solución automatiza el proceso completo con cuatro workflows de n8n que trabajan sobre una base de datos PostgreSQL, ambos ejecutándose en contenedores Docker:

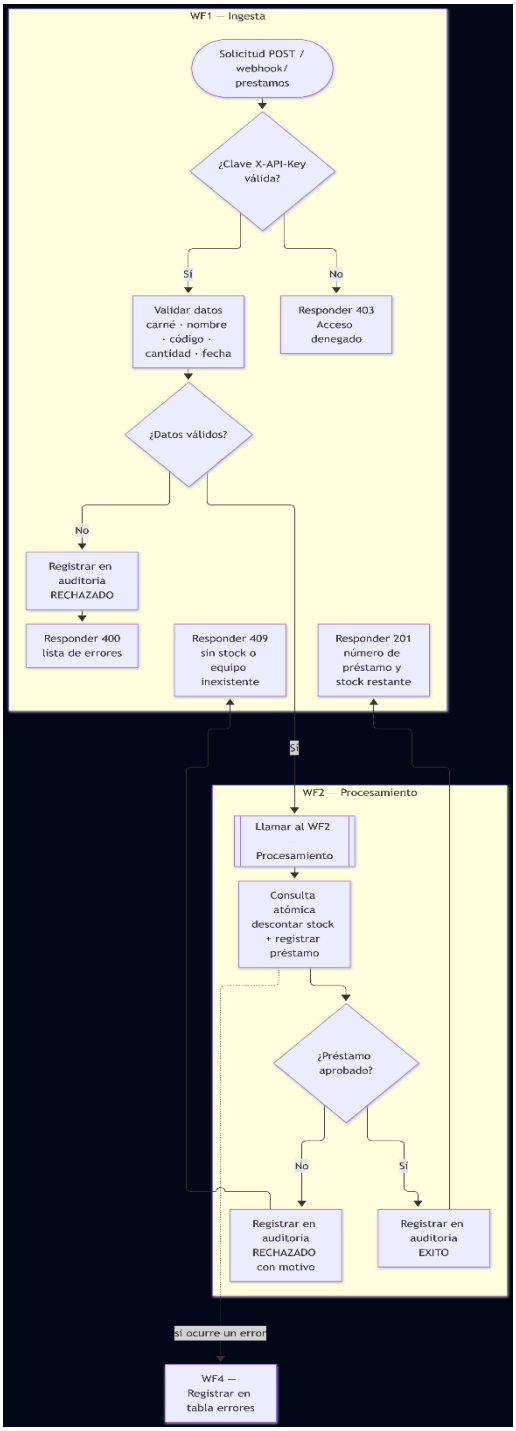


Figura 3. Diagrama del flujo principal (WF1 y WF2).

5.1 WF1 – Ingesta de préstamos

Objetivo: ser la única puerta de entrada al sistema. Ninguna solicitud llega a la base de datos sin pasar antes por este workflow.

Disparador: nodo Webhook que recibe peticiones POST en /webhook/prestamos. Exige la cabecera X-API-Key (Header Auth): sin la clave correcta, n8n responde 403 y el workflow ni siquiera se ejecuta.

Es la puerta de entrada del sistema. Recibe la solicitud por HTTP, comprueba la clave de acceso, valida los datos y decide si la rechaza o la envía a procesamiento.

3.1 Nodos

Nodo	Tipo	Función
Webhook	Webhook	Recibe POST /webhook/prestamos. Exige la cabecera X-API-Key mediante Header Auth.
Validar datos	Code	Limpia y valida carné, nombre, código de equipo, cantidad y fecha. Devuelve valido y la lista de errores.
¿Datos validos?	If	Separa el flujo en datos válidos e inválidos.
Auditoria rechazo	Postgres (Insert)	Registra la solicitud rechazada con sus errores.
Responder rechazo	Respond to Webhook	Responde con código 400 y la lista de errores.
Procesar en WF2	Execute Workflow	Envía los datos válidos al WF2 y espera su resultado.
Responder resultado	Respond to Webhook	Responde 201 si el préstamo se aprobó o 409 si no.

3.2 Reglas de validación

Las reglas de validación aplicadas son las siguientes:

Dato	Regla
carne	Formato 0000-00-0000 (expresión regular).
nombre_estudiante	Solo letras y espacios, de 3 a 100 caracteres.
codigo_equipo	Formato AAA-000, por ejemplo LAP-001.
cantidad	Número entero entre 1 y 10.
fecha_devolucion	Fecha real en formato AAAA-MM-DD, entre hoy y 30 días después.



Figura 4. WF1 – Ingesta de préstamos.

5.2 WF2 – Procesamiento de préstamos

Objetivo: decidir si el préstamo se puede entregar y, si se puede, registrarlo sin dejar el inventario inconsistente.

Disparador: nodo When Executed by Another Workflow. Solo se ejecuta cuando lo llama el WF1, por lo que nadie puede usarlo desde afuera para saltarse la validación.

Su nodo central ejecuta una única consulta SQL que descuenta el stock solo si alcanza y, en la misma instrucción, registra el préstamo. Al ser una sola operación, PostgreSQL la trata como una transacción: si dos personas piden el último equipo al mismo tiempo, solo una lo obtiene, y si algo falla no se guarda nada a medias. Los datos del estudiante viajan como parámetros separados del SQL, lo que evita la inyección SQL.

Solo puede ejecutarse desde el WF1, nunca directamente desde fuera. Su nodo central ejecuta una única consulta SQL que descuenta el stock solo si alcanza y, en la misma instrucción, registra el préstamo:

```
WITH equipo AS (
  UPDATE equipos SET stock_disponible = stock_disponible - $2
  WHERE codigo = $1 AND activo = TRUE AND stock_disponible >= $2
  RETURNING id_equipo, nombre, stock_disponible
), nuevo AS (
  INSERT INTO prestamos (carne, nombre_estudiante, id_equipo, cantidad,
                         fecha_devolucion_esperada)
  SELECT $3, $4, id_equipo, $2, $5::date FROM equipo
  RETURNING id_prestamo
)
SELECT ... AS aprobado, ... AS id_prestamo, ... AS equipo_existe ...;
```

Como todo ocurre en una sola instrucción, PostgreSQL la ejecuta como una transacción: si dos solicitudes compiten por la última unidad, solo una la obtiene, y si algo falla no se guarda nada a medias.

4.1 Nodos

Nodo	Tipo	Función
When Executed by Another Workflow	Execute Workflow Trigger	Recibe los datos validados desde el WF1.
Registrar prestamo	Postgres (Execute Query)	Consulta atómica parametrizada descrita arriba.
¿Prestamo aprobado?	If	Separa préstamos aprobados y rechazados.
Auditoria exito	Postgres (Insert)	Registra el préstamo aprobado con su número y el stock restante.
Resultado exito	Edit Fields	Arma la respuesta: estado APROBADO, número de préstamo, equipo, stock restante y fecha de devolución.
Auditoria sin stock	Postgres (Insert)	Registra el rechazo y su motivo: stock insuficiente o equipo inexistente.
Resultado rechazo	Edit Fields	Arma la respuesta: estado RECHAZADO, motivo y stock disponible.

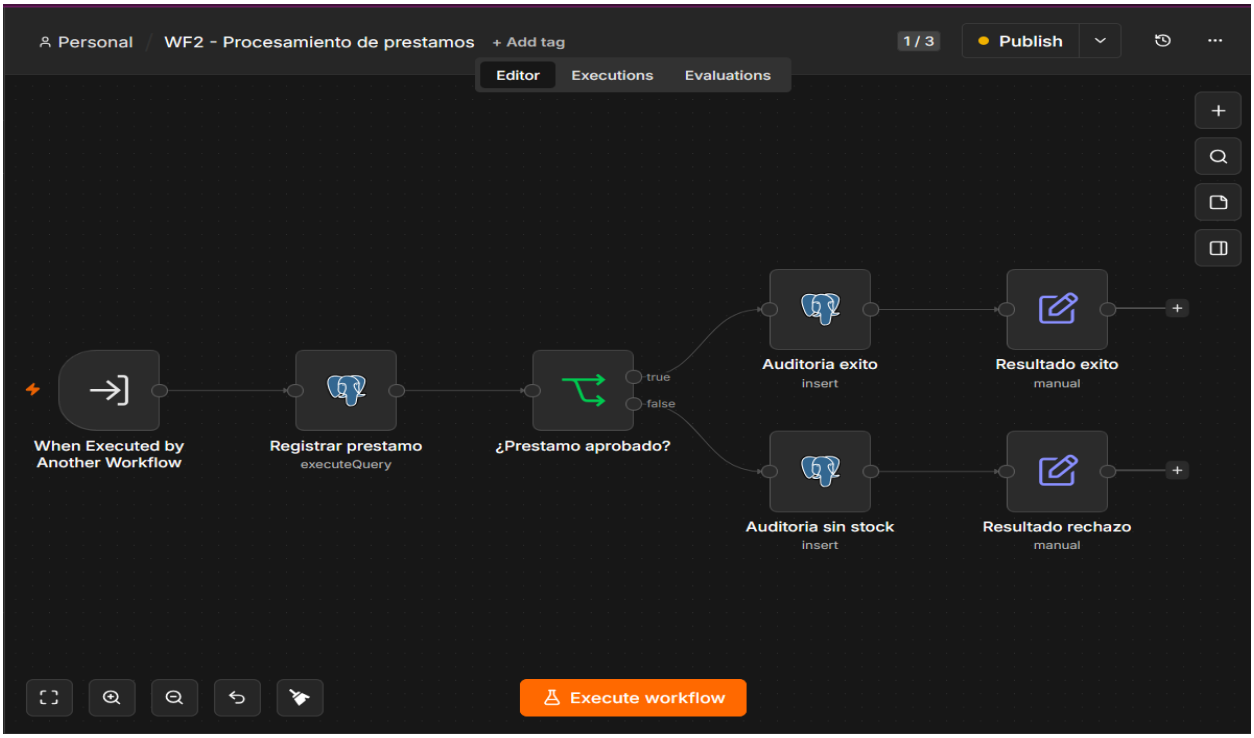


Figura 5. WF2 – Procesamiento de préstamos.

5.3 WF3 – Salida de reportes

Objetivo: dar seguimiento a los préstamos sin que nadie tenga que revisar el cuaderno: detectar los equipos que no se devolvieron a tiempo y dejar un reporte verificable.

Disparador: nodo Schedule Trigger que lo ejecuta todos los días a las 7:00 a. m. (hora de Guatemala).

El workflow marca como VENCIDO todo préstamo activo cuya fecha de devolución ya pasó y genera dos salidas: un registro con el reporte completo en la tabla auditoria y un archivo CSV en la carpeta reportes, con el nombre reporte_vencidos_AAAA-MM-DD.csv, que se puede abrir en Excel. Si un día no hay vencidos, el reporte se genera igual indicando "Sin préstamos vencidos".

Se ejecuta automáticamente todos los días a las 7:00 a. m. Marca como VENCIDO todo préstamo activo cuya fecha de devolución ya pasó y produce un reporte con dos salidas verificables: un registro en la tabla auditoria y un archivo CSV en la carpeta reportes/, con el nombre reporte_vencidos_AAAA-MM-DD.csv.

5.1 Nodos

Nodo	Tipo	Función
Cada dia 7am	Schedule Trigger	Inicia el workflow diariamente a las 7:00 a. m.
Buscar vencidos	Postgres (Execute Query)	Marca los vencidos y devuelve la lista con los días de atraso. Usa "Always Output Data" para continuar aunque no haya vencidos.
Generar reporte	Code	Resume el resultado: fecha, total de vencidos, nuevos del día y detalle.
Auditoria reporte	Postgres (Insert)	Guarda el reporte completo en la bitácora.
Preparar filas CSV	Code	Convierte el reporte en una fila por préstamo vencido.
Convertir a CSV	Convert to File	Genera el archivo CSV con encabezados.
Guardar CSV	Read/Write Files from Disk	Escribe el archivo en /reportes.

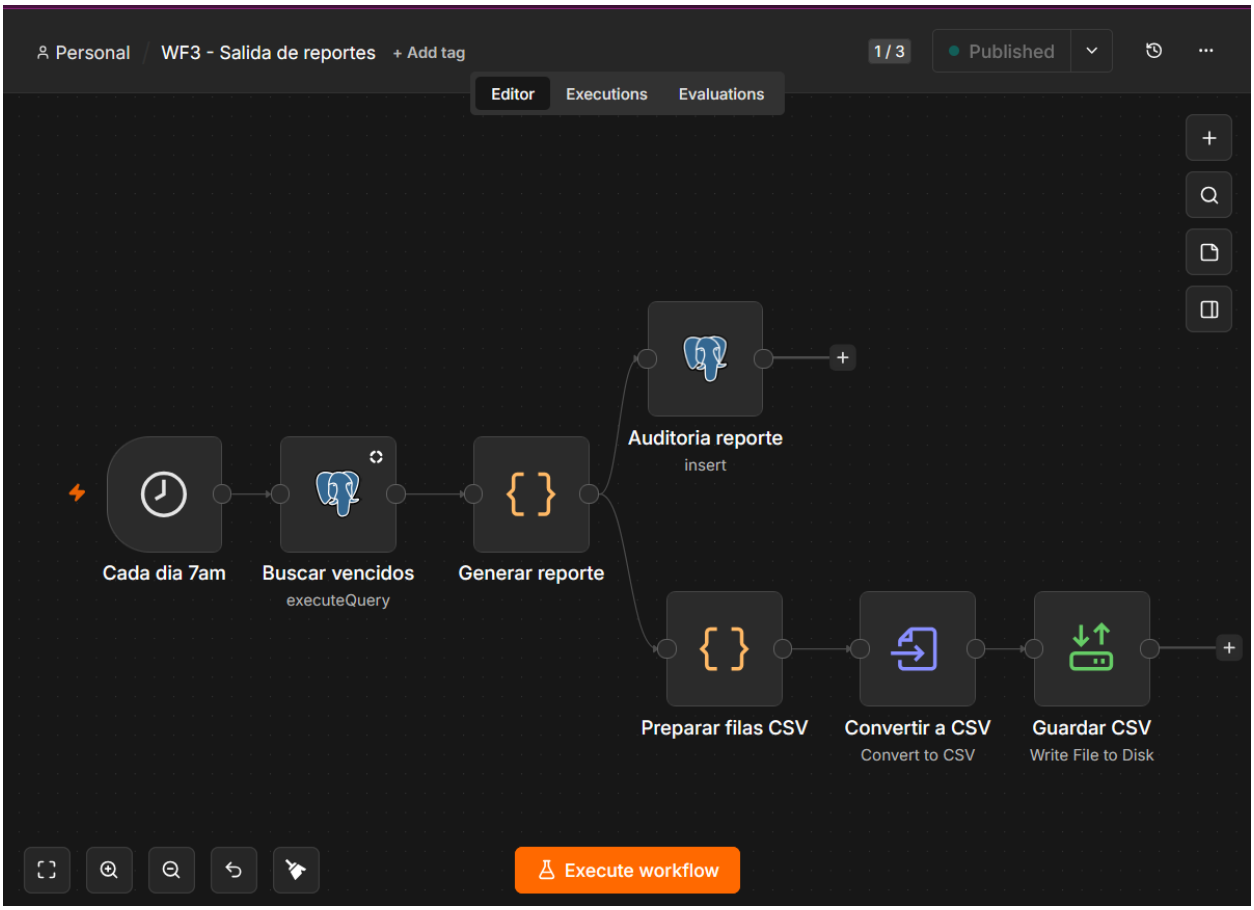


Figura 6. WF3 – Salida de reportes.

5.4 WF4 – Manejo de errores

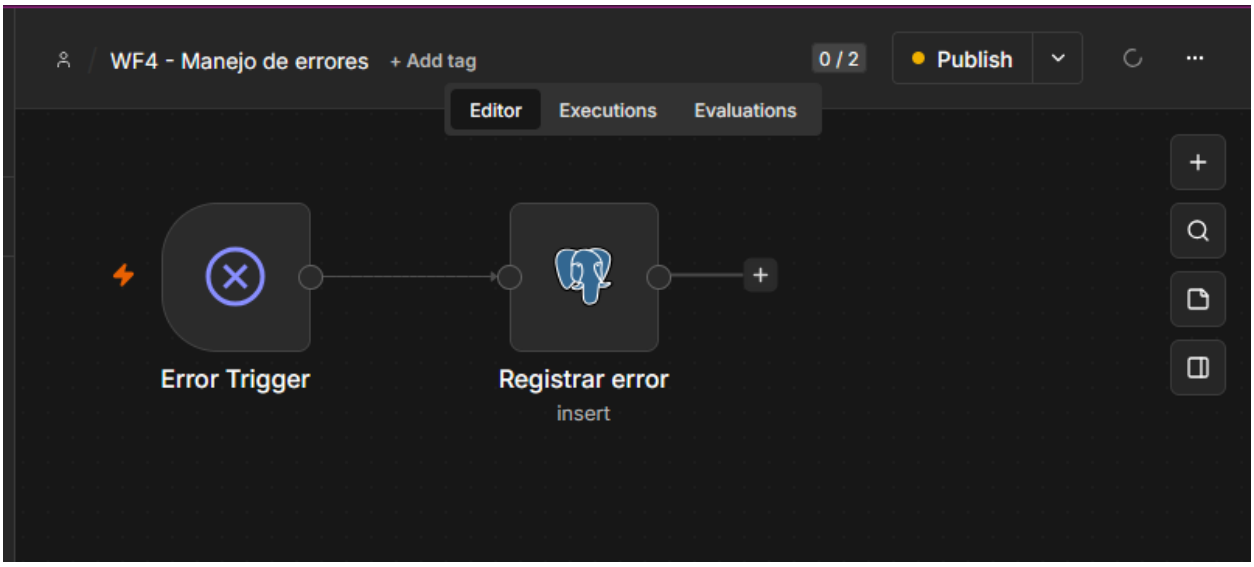
Objetivo: que ningún fallo pase desapercibido.

Disparador: nodo Error Trigger. Está configurado como Error Workflow del WF1, el WF2 y el WF3, así que n8n lo ejecuta automáticamente cuando cualquiera de ellos falla.

Registra en la tabla errores el workflow, el nodo exacto que falló, el mensaje, el número de ejecución y el detalle técnico. Como el WF1 llama al WF2, un fallo en el WF2 queda registrado dos veces: donde se originó y donde se recibió. Así se puede seguir el recorrido completo del error.

Está configurado como Error Workflow del WF1, el WF2 y el WF3. Cuando cualquiera de ellos falla en una ejecución de producción, n8n lo activa automáticamente con el detalle del error.

Nodo	Tipo	Función
Error Trigger	Error Trigger	Se activa cuando falla un workflow vigilado.
Registrar error	Postgres (Insert)	Guarda el workflow, el nodo que falló, el mensaje, el número de ejecución y el detalle técnico en la tabla errores.



5.5 Resumen de nodos utilizados

En total el sistema usa doce tipos de nodo distintos, el doble del mínimo de seis que pide el proyecto: Webhook, Code, If, Postgres, Respond to Webhook, Execute Workflow, Execute Workflow Trigger, Edit Fields, Schedule Trigger, Convert to File, Read/Write Files from Disk y Error Trigger.

6. Como Trabajan Juntos

1. Una solicitud llega al webhook del WF1 con la clave de acceso.
2. El WF1 valida los datos. Si son inválidos, registra el rechazo y responde 400.
3. Si son válidos, el WF1 llama al WF2, que intenta registrar el préstamo y descontar el stock.
4. El WF2 registra el resultado en la auditoría y lo devuelve al WF1, que responde 201 (aprobado) o 409 (sin stock o equipo inexistente).
5. Cada mañana, el WF3 revisa los préstamos vencidos y genera el reporte.
6. Si cualquiera de los tres falla, el WF4 registra el error.

Código	Significado
201	Préstamo aprobado y registrado.
400	Uno o más datos no cumplen las validaciones.
403	Falta la clave X-API-Key o es incorrecta.
409	El equipo no existe, está inactivo o no hay stock suficiente.

7. Manejo de errores y de los datos

7.1 Errores esperados

Los errores previsibles se manejavn dentro del propio flujo y se responden con el código HTTP estándar que corresponde, para que quien envía la solicitud sepa exactamente qué pasó:

Código	Situación	Registro
201	Préstamo aprobado y registrado.	auditoria: PRESTAMO_REGISTRADO / EXITO
400	Uno o más datos no cumplen las validaciones.	auditoria: SOLICITUD_PRESTAMO / RECHAZADO
403	Falta la clave X-API-Key o es incorrecta.	Rechazado por n8n antes de ejecutar el flujo
409	El equipo no existe, está inactivo o no hay stock suficiente.	auditoria: PRESTAMO_RECHAZADO / RECHAZADO

7.2 Errores inesperados

Los fallos no previstos (por ejemplo, que la base de datos no responda) los captura el WF4. Como el WF1 llama al WF2, un fallo en el WF2 queda registrado dos veces: en el nodo donde se originó y en el nodo del WF1 que lo recibió. Así se puede seguir el recorrido completo del error.

7.3 Consistencia de la información

La consistencia se garantiza en tres niveles: la validación en el WF1 impide que entren datos mal formados; la consulta atómica del WF2 impide que el stock y los préstamos queden desalineados; y las restricciones CHECK de PostgreSQL actúan como última barrera aunque un workflow tuviera un defecto.

8. Consideraciones de seguridad

Medida	Implementación	Riesgo que reduce
Credenciales en variables de entorno	Usuario y contraseña de PostgreSQL y clave de cifrado de n8n en .env, leídas por docker-compose.yml. El .env está en .gitignore y el repositorio solo incluye .env.example.	Exposición de contraseñas en el código o en GitHub.
Control de acceso	El webhook exige la cabecera X-API-Key (Header Auth). Sin la clave correcta n8n responde 403.	Solicitudes de terceros no autorizados.
Validación de entradas	Expresiones regulares y rangos para cada campo antes de tocar la base de datos.	Datos corruptos e inyección de contenido malicioso.
Consultas parametrizadas	Los datos viajan separados del SQL ($1…$5) o mediante el modo Insert de n8n.	Inyección SQL.
Credenciales cifradas en n8n	N8N_ENCRYPTION_KEY cifra las credenciales guardadas. Los workflows exportados solo contienen su nombre.	Robo de credenciales desde la configuración de n8n.
Acceso a archivos restringido	N8N_RESTRICT_FILE_ACCESS_TO limita a n8n a la carpeta /reportes.	Lectura o escritura de archivos del sistema.
Sub-workflow no expuesto	El WF2 solo se ejecuta cuando lo llama el WF1.	Registro de préstamos saltándose la validación.
Datos personales mínimos	Solo se guardan el carné y el nombre, lo indispensable para el préstamo.	Exposición innecesaria de información de estudiantes.
Clave fuera del código (aplicación web)	La aplicación web lee la clave del webhook solo de la variable de entorno N8N_API_KEY. Si falta, se niega a arrancar.	Exposición de la clave en el código o en GitHub.
Validación en dos capas	La aplicación web aplica las mismas reglas del WF1 antes de llamar a n8n.	Datos que intenten saltarse una sola validación.
Tiempo máximo de espera	La aplicación corta la conexión con n8n a los 10 segundos y muestra un mensaje claro.	Bloqueo de la aplicación si n8n deja de responder.

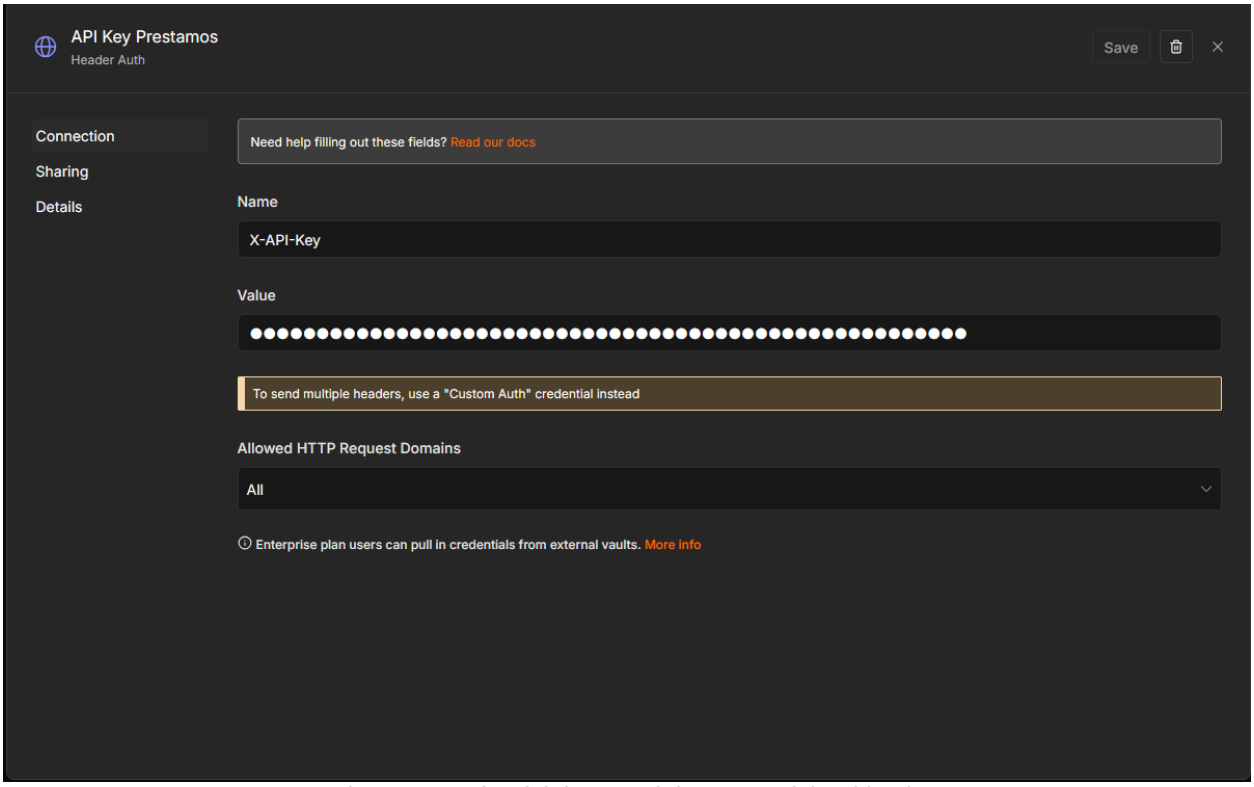


Figura 7. Credencial de control de acceso del webhook.

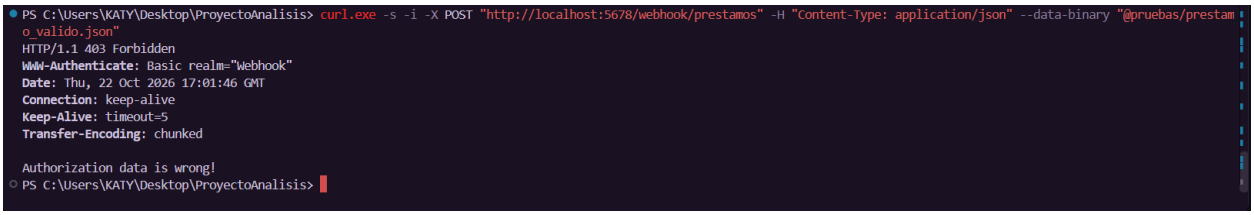


Figura 8. Solicitud rechazada por falta de clave de acceso.

9. Pruebas realizadas

Las pruebas se ejecutaron entre el 21 y el 22 de octubre de 2026. Los archivos de entrada están en la carpeta pruebas/ del repositorio.

#	Caso	Entrada	Resultado esperado	Resultado obtenido
1	Datos inválidos	prestamo_invalido.json (5 campos incorrectos)	400 con 5 errores; auditoría RECHAZADO	Correcto
2	Préstamo válido	prestamo_valido.json (1 proyector PRO-001)	201; stock de 2 a 1	Correcto
3	Segundo préstamo válido	prestamo_valido.json	201; stock de 1 a 0	Correcto

#	Caso	Entrada	Resultado esperado	Resultado obtenido
4	Sin stock	prestamo_valido.json	409 "Stock insuficiente"	Correcto
5	Consistencia	Consulta a equipos, prestamos y auditoria	2 prestados + 0 disponibles = 2 totales; 4 eventos en auditoría	Correcto
6	Error simulado	Tabla prestamos renombrada temporalmente	WF4 registra el error en WF2 y en WF1; datos intactos	Correcto
7	Préstamo vencido	crear_vencido.sql y ejecución del WF3	Préstamo pasa a VENCIDO (7 días); CSV generado	Correcto
8	Acceso sin clave	Solicitud sin cabecera X-API-Key	403	[Completar]



Figura 9. Préstamo aprobado.



Figura 10. Solicitud con datos inválidos.

```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> docker exec -it inventario_db psql -U irma_admin -d inventario_lab -c "SELECT id_auditoria, fecha, workflow, accion, resultad
o FROM auditoria ORDER BY id_auditoria;"
 id_auditoria |          fecha          |            workflow             |       accion        | resultado
--------------+-------------------------+---------------------------------+---------------------+-----------
            1 | 2026-10-21 17:24:40.537014 | WF1 - Ingesta de prestamos      | SOLICITUD_PRESTAMO  | RECHAZADO
            2 | 2026-10-21 22:41:31.937784 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            3 | 2026-10-21 22:43:56.446106 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            4 | 2026-10-21 22:45:40.536453 | WF2 - Procesamiento de prestamos | PRESTAMO_RECHAZADO  | RECHAZADO
            5 | 2026-10-22 08:29:04.570097 | WF3 - Salida de reportes        | REPORTE_VENCIDOS    | EXITO
            6 | 2026-10-22 08:53:54.301595 | WF3 - Salida de reportes        | REPORTE_VENCIDOS    | EXITO
            7 | 2026-10-22 14:02:21.805519 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            8 | 2026-10-22 14:03:26.065634 | WF1 - Ingesta de prestamos      | SOLICITUD_PRESTAMO  | RECHAZADO
(8 rows)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug inventario_db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```

Figura 11. Bitácora de auditoría.

```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> docker exec -it inventario_db psql -U irma_admin -d inventario_lab -c "SELECT id_error, fecha, workflow, nodo, mensaje FROM e
rrores;"
 id_error |          fecha          |            workflow             |       nodo       |                mensaje
----------+-------------------------+---------------------------------+------------------+---------------------------------------
        1 | 2026-10-21 23:17:30.676317 | WF2 - Procesamiento de prestamos | Registrar prestamo | relation "prestamos" does not exist
        2 | 2026-10-21 23:17:30.687704 | WF1 - Ingesta de prestamos      | Procesar en WF2  | relation "prestamos" does not exist
(2 rows)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug inventario_db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```

Figura 12. Errores registrados por el WF4.

```
reportes > reporte_vencidos_2026-10-22.csv
1   fecha_reporte,id_prestamo,carne,estudiante,equipo,cantidad,fecha_devolucion,dias_atraso
2   2026-10-22,3,4090-23-5678,Carlos Caal Tiul,LAP-001 - Laptop Dell Latitude,1,2026-10-15,7
```

Figura 13. Reporte CSV de préstamos vencidos.

10. Instalación y reproducibilidad

El archivo README.md del repositorio contiene las instrucciones completas. En resumen:

1. Instalar Docker Desktop y Git.
2. Clonar el repositorio y copiar .env.example como .env, completando sus valores.
3. Ejecutar docker compose up -d. PostgreSQL crea las tablas automáticamente.
4. Entrar a http://localhost:5678, crear la cuenta de n8n y las credenciales Postgres Inventario y API Key Prestamos.
5. Importar los workflows de la carpeta workflows/ en el orden WF04, WF02, WF01 y WF03, asignar credenciales, configurar el WF4 como Error Workflow y publicar.
6. Ejecutar los casos de la carpeta pruebas/.

7. Para la aplicación web: configurar la variable de entorno N8N_API_KEY y ejecutar mvn spring-boot:run dentro de app-prestamos.
8. El historial de commits del repositorio documenta la evolución del proyecto, desde el entorno inicial (v0.2) hasta la versión con diagramas (v0.6).

11. Aplicación web de préstamos

Para que el personal del laboratorio registre préstamos sin usar la terminal, se desarrolló una aplicación web en el módulo app-prestamos del repositorio. La aplicación no reemplaza a los workflows: es una nueva puerta de entrada que envía cada solicitud al webhook del WF1, de modo que la lógica, la auditoría y el manejo de errores de n8n se siguen aplicando.

11.1 Tecnologías y arquitectura

Se construyó con Spring Boot 4.1, que incluye su propio servidor web (Tomcat) en el puerto 8080, y con Thymeleaf como motor de plantillas HTML. Sigue el patrón Modelo–Vista–Controlador (MVC): el controlador recibe el formulario, un servicio aplica las reglas de negocio y la vista muestra el resultado. Las clases reciben sus dependencias por inyección de dependencias, lo que permite reemplazar cualquier pieza por una simulación durante las pruebas.

Clase	Responsabilidad
PrestamoController	Atiende GET y POST de /prestamos y elige la vista que se muestra.
FormularioPrestamo	Recibe los campos del formulario como texto, para que cualquier dato mal escrito llegue al validador y reciba un mensaje claro.
ServicioPrestamos	Valida la solicitud y solo si es válida la envía a n8n. Convierte los fallos de comunicación en un error controlado (503).
ValidadorPrestamo	Aplica las mismas cinco reglas del nodo Validar datos del WF1.
PrestamoGateway y N8nPrestamoGateway	Interfaz y cliente HTTP (RestClient) que llama al webhook con la cabecera X-API-Key e interpreta las respuestas 201, 400, 403 y 409.
ConfiguracionPrestamos	Crea y conecta las piezas, y lee la URL del webhook y la clave desde la configuración.
prestamo.html	Plantilla Thymeleaf con el formulario y el resultado en verde, naranja o rojo.

11.2 Flujo de una solicitud

- El usuario llena el formulario en http://localhost:8080/prestamos y lo envía.
- El controlador convierte el formulario en una solicitud y la entrega al servicio.
- El servicio la valida. Si es inválida, responde de inmediato con la lista de errores, sin contactar a n8n.
- Si es válida, el gateway la envía al webhook del WF1, que la procesa junto con el WF2.
- La respuesta se muestra en la página: verde si se aprobó, naranja si se rechazó y rojo si n8n no respondió. Si se rechaza, los datos se conservan para que el usuario solo corrija lo necesario.

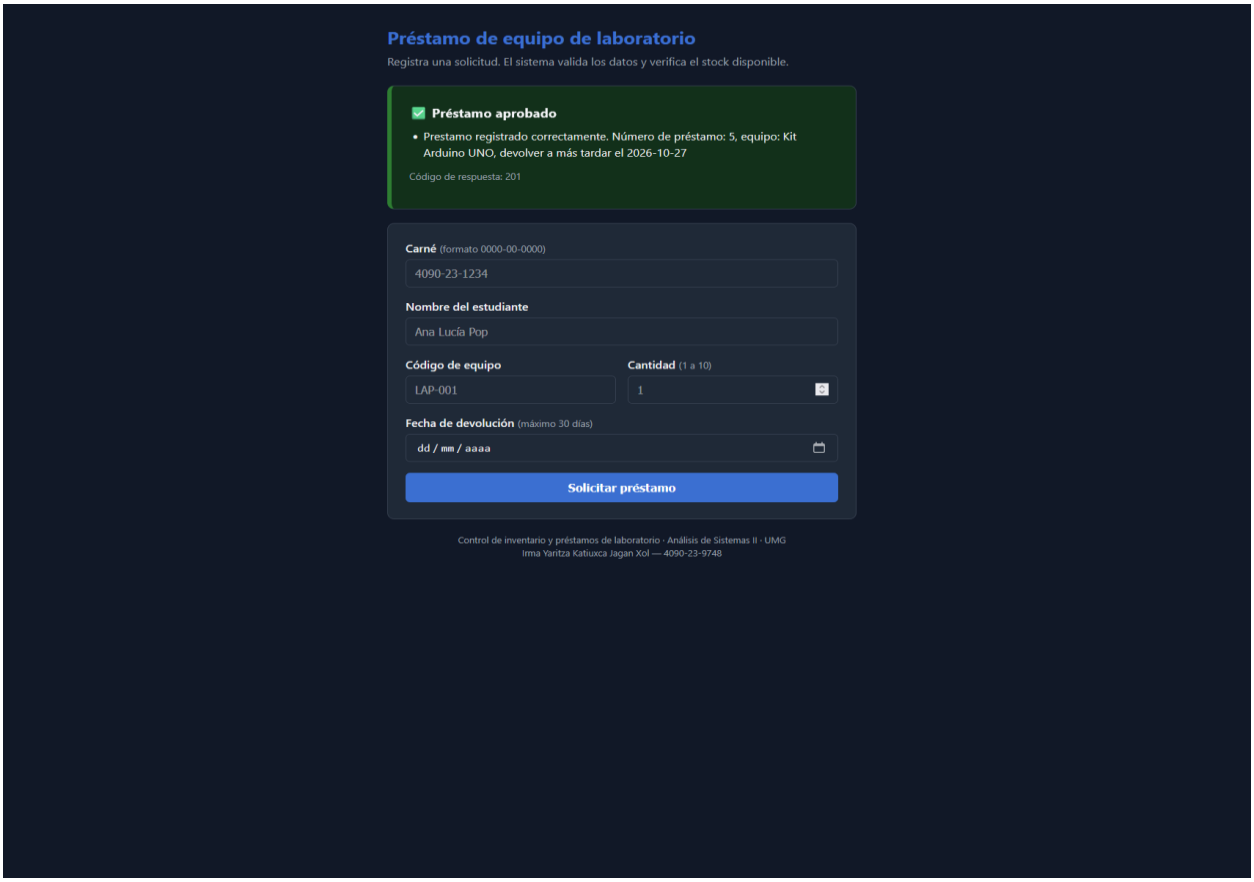


Figura 14. Préstamo aprobado desde la aplicación web.

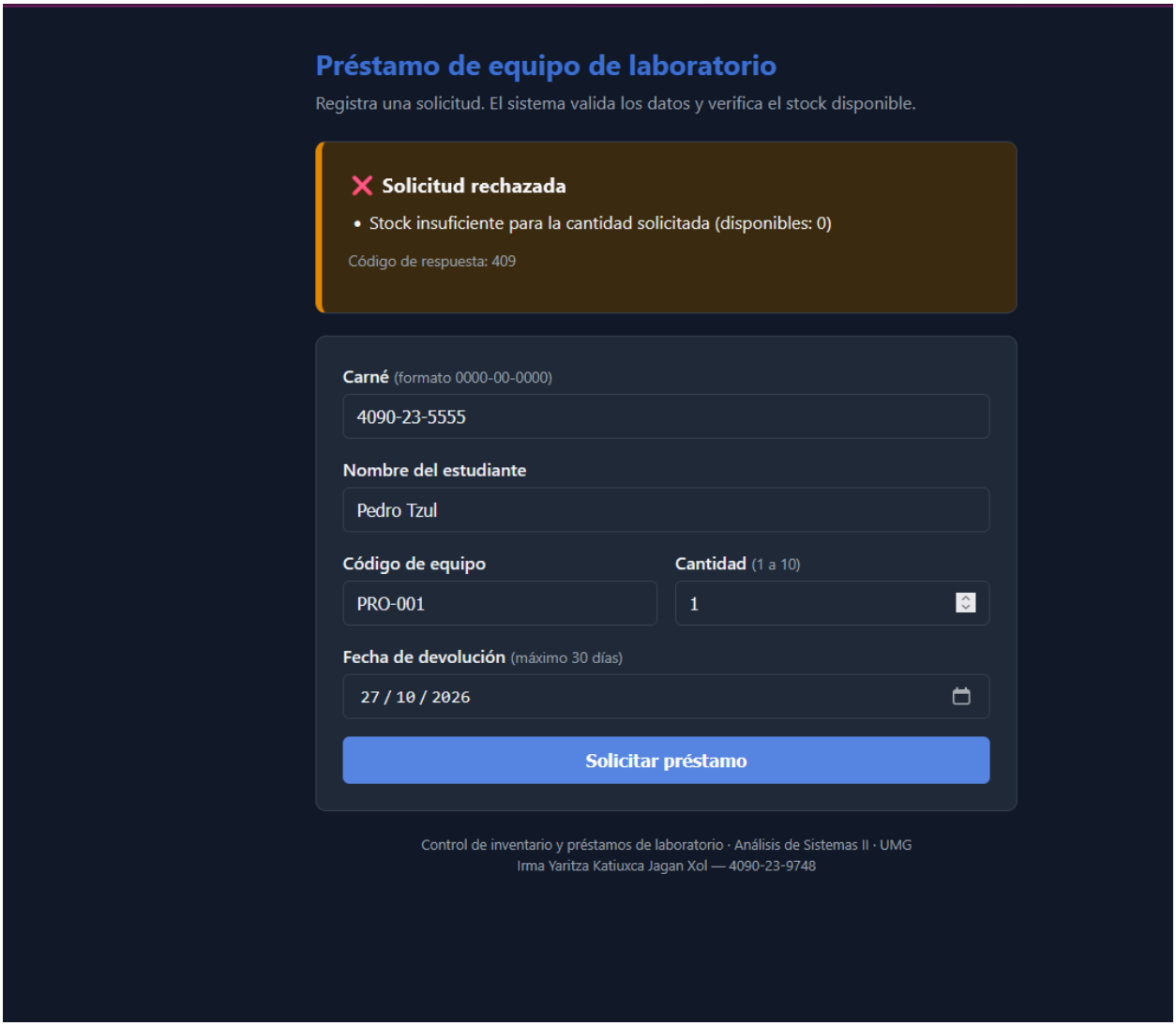


Figura 15. Solicitud rechazada desde la aplicación web.

11.3 Seguridad de la aplicación

- La clave del webhook se lee únicamente de la variable de entorno N8N_API_KEY. Si no está configurada, la aplicación se niega a arrancar e indica qué falta, en lugar de funcionar a medias.
- La clave vive solo en el servidor: el navegador nunca la recibe.
- La validación se aplica en dos capas, en la aplicación y en el WF1, de modo que nadie puede saltarse las reglas enviando datos directamente al webhook.
- La conexión con n8n tiene un tiempo máximo de espera de 10 segundos. Si se supera, el usuario recibe un mensaje claro en lugar de una página bloqueada.

12. Pruebas unitarias automatizadas

Además de las pruebas del sistema de la sección 8, la aplicación web cuenta con 82 pruebas unitarias automatizadas. Se ejecutan con un solo comando (mvn verify), tardan pocos segundos y no necesitan Docker ni n8n, porque las dependencias externas se reemplazan por simulaciones.

12.1 Herramientas

Herramienta	Uso en el proyecto
JUnit 5	Motor de pruebas. Se usaron pruebas parametrizadas para probar cada regla con varios valores válidos, inválidos y límite.
Mockito	Simula el gateway de n8n para probar el servicio, y el servicio para probar el controlador (@MockitoBean).
MockRestServiceServer	Simula el servidor HTTP de n8n y verifica que la petición lleve la URL, el método, la clave y el JSON correctos.
MockMvc	Simula el navegador: abre la página, envía el formulario y revisa el HTML generado.
JaCoCo	Mide la cobertura del código y hace fallar la construcción si la cobertura de líneas baja del 80 %.

Para que las pruebas que dependen de la fecha den siempre el mismo resultado, el validador recibe un reloj (Clock) que en las pruebas se fija en el 22 de octubre de 2026.

12.2 Casos probados

Clase de prueba	Pruebas	Qué verifica
ValidadorPrestamoTest	48	Las cinco reglas con valores válidos, inválidos y límite: por ejemplo, cantidades 1 y 10 aceptadas, 0 y 11 rechazadas, y la fecha 2026-02-30 rechazada por no existir.
ServicioPrestamosTest	10	Que los datos inválidos nunca lleguen a n8n, que los válidos se envíen normalizados una sola vez y que una caída de n8n se convierta en un error 503 controlado.
N8nPrestamoGatewayTest	14	La petición HTTP exacta y la interpretación de cada respuesta de n8n: 201, 400, 403, 409, 500, respuestas vacías y servidor apagado.
PrestamoControllerTest	9	Redirección, formulario vacío, colores del resultado, conservación de los datos al rechazar y cantidades no numéricas.
AppPrestamosApplicationTest	1	Que la aplicación completa arranca y conecta el controlador, el servicio y el gateway real.
Total	**82**	**Todas aprobadas.**

12.3 Cobertura de código

Métrica (JaCoCo)	Resultado
Instrucciones	661 de 666 (99 %)
Ramas de decisión	63 de 64 (98 %)
Líneas	148 de 150
Métodos	45 de 46
Clases	12 de 12

Lo único sin cubrir es el método main que arranca la aplicación, que en las pruebas se inicia por otro camino, y una rama de un caso límite. Toda la lógica de negocio, el gateway y el controlador están cubiertos, muy por encima del mínimo de 80 % configurado.

Las pruebas también ayudaron a detectar y corregir un defecto real: al solicitar un equipo inexistente, la página mostraba «(disponibles: 0)», un dato que confundía al usuario. Se corrigió el mensaje y se agregó una prueba que reproduce ese caso para que no vuelva a aparecer.

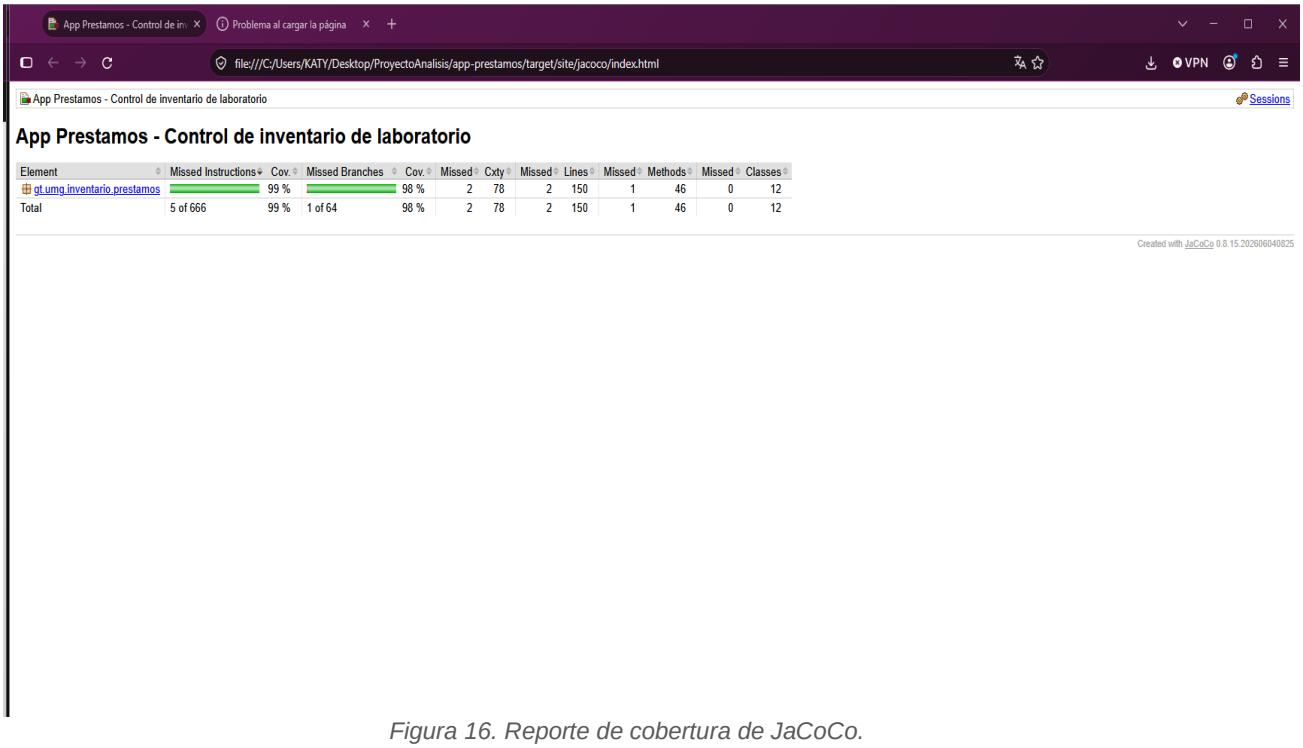


Figura 16. Reporte de cobertura de JaCoCo.

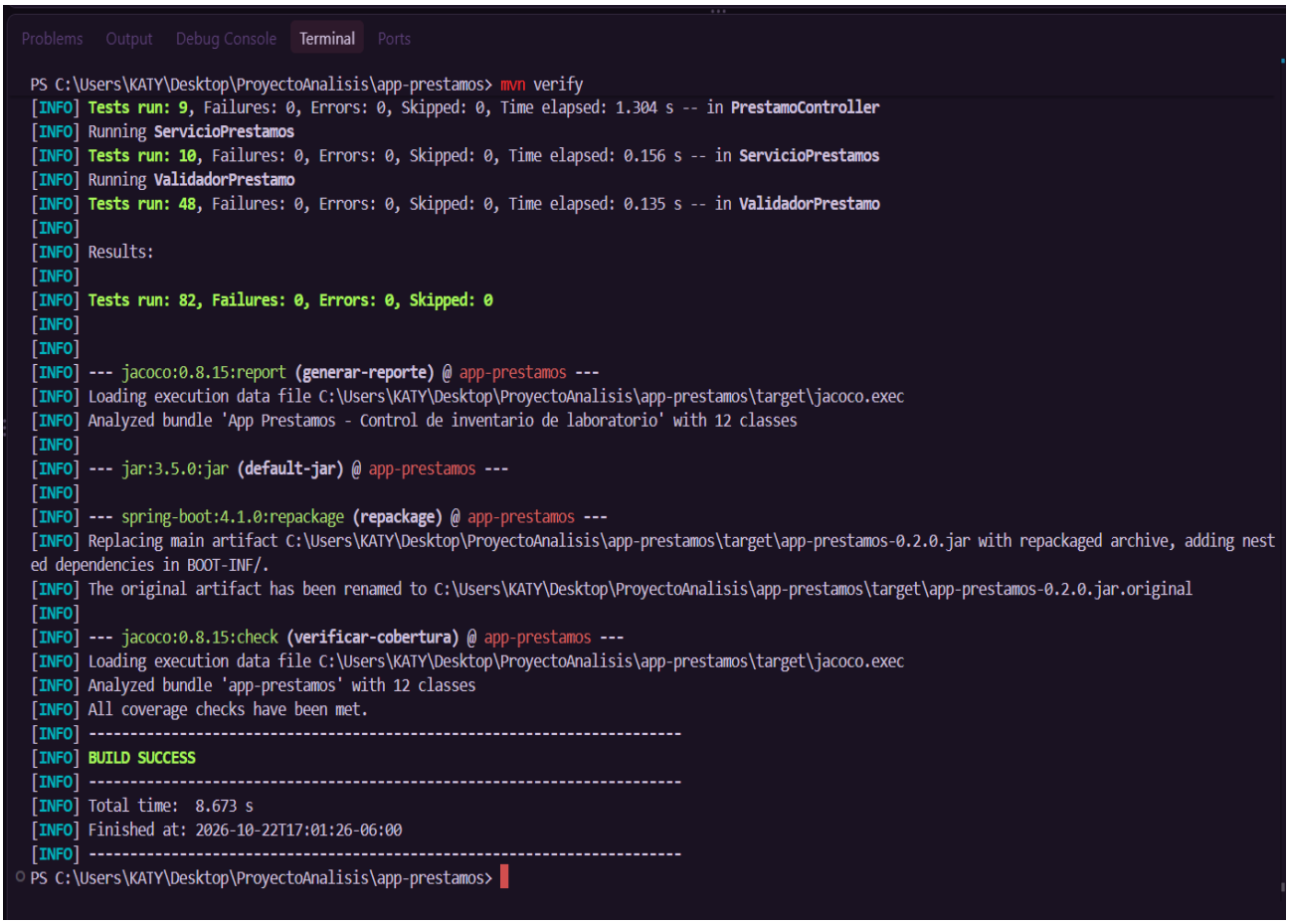


Figura 17. Ejecución de las pruebas unitarias.

13. Conclusiones

- La automatización con n8n permitió reemplazar un proceso manual por uno que valida, registra y reporta sin intervención humana, y que deja evidencia de cada operación.
- Delegar la consistencia del inventario a la base de datos, mediante una operación atómica y restricciones de integridad, resultó más confiable que comprobarla en pasos separados dentro del workflow.
- Separar la solución en workflows con una sola responsabilidad simplificó las pruebas y permitió que el manejo de errores rastreara exactamente dónde se originó cada fallo.
- La seguridad se integró desde el diseño (variables de entorno, control de acceso, validación y consultas parametrizadas) en lugar de agregarse al final.
- Repetir en la aplicación web las mismas reglas del WF1 creó una validación en dos capas: los errores se detectan antes de llegar a n8n y nadie puede saltarse las reglas.
- Las pruebas unitarias automatizadas permitieron modificar el código con confianza: cada cambio se verificó en segundos con 82 pruebas, y una de ellas ayudó a corregir un defecto antes de la entrega.

13.1 Trabajo futuro

- Workflow de devolución que registre fecha_devolucion_real y restituya el stock.
- Notificaciones por correo a los estudiantes con préstamos vencidos.
- Autenticación de usuarios en la aplicación web, para registrar quién aprueba cada préstamo.
- Pruebas de integración automatizadas que levanten n8n y PostgreSQL en contenedores durante la ejecución de las pruebas.

14. Referencias y recursos de apoyo

- n8n. Documentación oficial. https://docs.n8n.io
- PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
- Docker Inc. Docker Compose Documentation. https://docs.docker.com/compose/
- Mermaid. Documentación de diagramas. https://mermaid.js.org
- Universidad Mariano Gálvez de Guatemala. Guía Completa del Proyecto Final – Análisis de Sistemas II.
- Spring. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
- Thymeleaf. Tutorial: Using Thymeleaf. https://www.thymeleaf.org/documentation.html
- JUnit Team. JUnit User Guide. https://docs.junit.org/
- Mockito. Documentación oficial. https://site.mockito.org
- JaCoCo. Java Code Coverage Library. https://www.jacoco.org/jacoco/
- Se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo en la guía técnica paso a paso y en la redacción de este documento, conforme al compromiso de dar a conocer los recursos utilizados.